CardioIA: Da Construção de Módulos à Solução Integrada

Relatório técnico da entrega final · Santo André, novembro de 2026

Integrantes: Vitor Eiji, Beatriz Pilecarte, Franciscmar Alves, Matheus Soares e Antonio Barros. Tutor: Caique Nonato. Coordenador: Andre Godoi Chiochiovatto.

1. Resumo

O CardioIA integra, em uma única plataforma, os módulos construídos nas fases anteriores: o modelo preditivo de risco de crise cardíaca (Fase 6) e a triagem conversacional de sintomas (Fase 5). Um sensor simulado em MicroPython (ESP32, no Wokwi) envia frequência cardíaca, temperatura e saturação de oxigênio a um backend em Python, que calcula o risco, aciona protocolos clínicos e devolve a recomendação. O resultado aparece em três lugares quase ao mesmo tempo: no próprio sensor (LEDs, buzzer e display OLED), em uma aplicação web e em um aplicativo Android. O sistema é um protótipo acadêmico, com dados sintéticos, e não substitui avaliação médica.

2. Links da entrega

Item	Endereço
Web (Vercel)	https://fase-7-da-construcao-de-modulos-a-s.vercel.app
API (Render)	https://fase-7-da-construcao-de-modulos-a.onrender.com
Sensor (Wokwi)	https://wokwi.com/projects/479615874964743169
App Android (.apk)	[colar o link do build do EAS]
Vídeo (até 5 min)	[colar o link do vídeo]
Repositório	[colar o link do repositório no GitHub]

A API roda no plano gratuito do Render e hiberna sem uso. A primeira chamada pode levar cerca de um minuto; abrir `/health` antes da demonstração acorda o serviço.

3. Arquitetura da solução

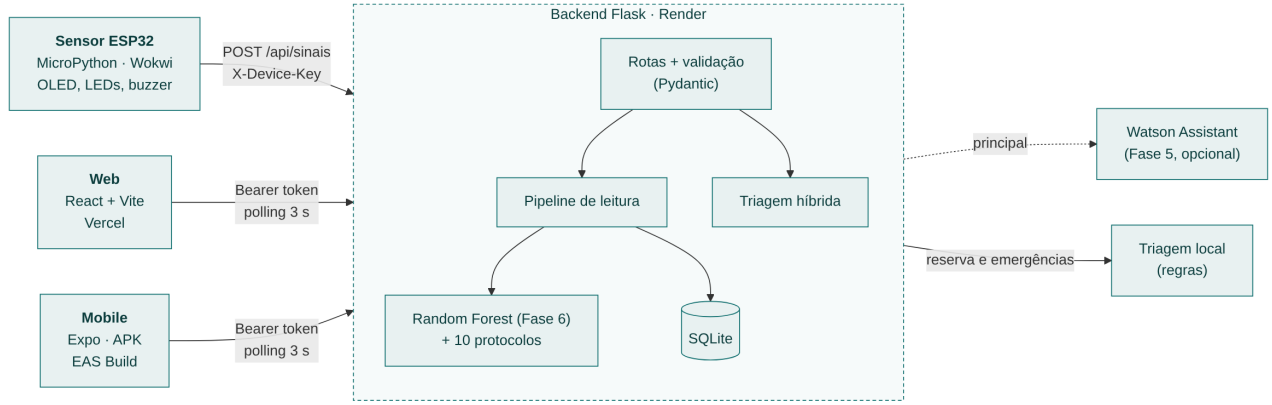


Figura 1. Visão geral: sensor, web e mobile conversam com um único backend, que concentra modelo, protocolos, banco e triagem.

Fluxo principal. O sensor faz `POST /api/sinais` com a chave do dispositivo. O backend valida a entrada, completa as 10 variáveis esperadas pelo modelo, calcula a probabilidade de crise, classifica o risco (BAIXO, MODERADO ou ALTO), aciona os protocolos, grava a leitura e responde com a recomendação. Web e app consultam a última leitura a cada 3 segundos (*polling*), solução simples e compatível com o plano gratuito de hospedagem.

Reaproveitamento. Da Fase 6 vêm o modelo Random Forest, as 10 variáveis e os 10 protocolos clínicos. Da Fase 5 vêm o fluxo de triagem do Watson Assistant, portado também para regras locais, e o app Expo de chat, que virou a aba Triagem do aplicativo.

4. Backend integrador (Python)

Aplicação Flask organizada em rotas, serviços e módulo de aprendizado de máquina, com Pydantic na validação, SQLite para o histórico e gunicorn em produção (1 processo e 4 threads, porque as sessões do chat ficam em memória).

Modelo preditivo e protocolos

- **Random Forest** (scikit-learn) treinado no build com dados sintéticos e semente fixa. Probabilidade ≥ 0,70 é ALTO; ≥ 0,30 é MODERADO.
- **Rótulo de treino:** exige pelo menos 2 de 6 critérios (carga > 80, espera > 60, SpO2 < 92, FC > 110, eventos > 4, recursos < 40). Por isso um único sinal alterado não chega a ALTO; FC alta e SpO2 baixa juntas, sim (exemplo validado em produção: FC 130 e SpO2 85 resultaram em ALTO, 82,5%, com 3 protocolos).
- **10 protocolos clínicos** da Fase 6 são acionados conforme as leituras, inclusive quando o risco geral fica abaixo de ALTO.
- **Completude das variáveis:** idade vem do cadastro do paciente, eventos em 24 h são contados no histórico e os indicadores de operação hospitalar vêm de `/api/operacao`.

Triagem conversacional híbrida

- O **Watson Assistant** é o motor principal quando as credenciais estão configuradas. Um cliente HTTP próprio foi usado no lugar do SDK, cuja versão atual exige parâmetros que o assistente da Fase 5 não tem.
- Uma **máquina de estados local**, portada dos 6 passos da *Action*, responde se o Watson falhar, demorar ou não estiver configurado. Um *circuit breaker* suspende o Watson por 60 s após 3 falhas seguidas.
- **Emergências** (por exemplo, dor no peito) são decididas localmente antes de qualquer chamada externa, e a interface mostra o alerta com o botão para ligar 192.
- O campo `origem` da resposta informa se quem respondeu foi o Watson ou a triagem local.

Segurança e robustez

- Tokens assinados com validade de 12 h para médico e paciente; o paciente enxerga apenas os próprios dados. O sensor usa chave própria (`X-Device-Key`), que não é exposta à web nem ao app.
- Validação estrita (limites de FC, temperatura, SpO2 e pressão, rejeição de NaN e infinito), corpo limitado a 64 KB, sessões de chat limitadas e expiradas, e erros sempre em JSON.
- Segredos apenas em variáveis de ambiente do Render e da Vercel; nada sensível no repositório.

5. Web, mobile e sensor

Aplicação web (React + Vite, Vercel)

Painel com login. O **médico** vê a lista de pacientes, o detalhe do risco com os protocolos acionados, o histórico em gráfico e a operação hospitalar; o **paciente** abre direto no próprio risco. A aba de triagem usa o mesmo backend. A publicação usa `vercel.json` e a variável `VITE_API_URL`, e o deploy é refeito a cada *push*.

Aplicativo mobile (Expo, EAS)

App em React Native com expo-router: login, painel com atualização a cada 3 segundos (pausa quando o app vai ao segundo plano), triagem por chat e aba Hospital exclusiva do perfil médico. A sessão fica no *SecureStore* do aparelho. O .apk é gerado pelo EAS Build (perfil `preview`), com a URL da API embutida em tempo de build.

Sensor (MicroPython no Wokwi)

ESP32 com potenciômetros simulando FC, temperatura e SpO2. A cada ciclo envia a leitura, mostra no OLED o risco e a probabilidade e aciona LEDs e buzzer (880 Hz, dois bipes, no ALTO). Há **duas tentativas de envio** e uma **classificação local de reserva** (FC > 110 ou SpO2 < 92) para o caso de a API estar fora do ar ou ainda acordando.

6. Atendimento aos requisitos da Fase 7

Requisito	Situação	Evidência
Backend integrador em Python (preditivo + conversacional)	Concluído	Render, 134 testes automatizados
Web em React + Vite na Vercel, com vercel.json	Concluído	URL da seção 2
Mobile em Expo com build .apk pelo EAS	[atualizar: concluído / em andamento]	Link do .apk na seção 2
Sensor em MicroPython no Wokwi com LED e OLED	Concluído	Projeto Wokwi, 14 testes de lógica
Diagrama de arquitetura	Concluído	Figura 1
Relatório em PDF (até 5 páginas)	Concluído	Este documento
Vídeo de até 5 minutos	[atualizar]	Link na seção 2

7. Resultados e validação

- **Ponta a ponta validado em produção:** Wokwi → API no Render → painel na Vercel, com a leitura aparecendo em segundos. FC 130 e SpO2 85 resultam em ALTO, e o sensor acende o LED vermelho, toca o buzzer e exibe o alerta no OLED.
- **Testes automatizados:** 134 no backend (pytest) e 14 na lógica do sensor, executados também em instalação limpa.
- **Perfis:** verificado que o paciente não vê a lista de pacientes nem a aba Hospital, e que o médico vê tudo.
- **Triagem:** a frase “dor no peito” aciona o alerta de emergência na web e no app; frases fora do vocabulário voltam ao menu local em vez de uma resposta vazia.
- **Consistência:** o percentual exibido pelo front e pelo backend foi alinhado (arredondamento do mesmo valor), evitando 80% em um lugar e 81% em outro.

8. Decisões técnicas, limitações e trabalhos futuros

Decisões

- **Triagem híbrida** em vez de depender só do Watson: a parte crítica (emergência) nunca depende de serviço externo, e a demonstração funciona mesmo sem as credenciais.
- **Polling** em vez de WebSocket: menos infraestrutura e comportamento previsível no plano gratuito.
- **Deploy a partir de um fork**, porque o repositório do projeto fica em uma organização e as plataformas de hospedagem exigem acesso ao repositório.

Limitações

- Dados e modelo sintéticos: a probabilidade reproduz a regra usada no treino e não tem valor clínico.
- A triagem local reconhece palavras e padrões, sem NLP, e não interpreta negações (“não tenho dor no peito” aciona a emergência, por escolha de segurança).
- O Render gratuito hiberna e tem disco efêmero: o banco SQLite é recriado com dados de demonstração a cada reinício, e as sessões de chat se perdem (o cliente abre outra sozinho).

Trabalhos futuros

Treino com dados reais anonimizados e validação clínica, banco gerenciado, notificações push para o médico, WebSocket para atualização instantânea e interpretação de negações na triagem.

9. Acesso para avaliação

Para evitar expor senhas em repositório público, as credenciais de demonstração constam apenas neste relatório.

Perfil	Usuário	Senha
Médico	medico	[preencher com MEDICO_SENHA]
Paciente	paciente	[preencher com PACIENTE_SENHA]

Roteiro rápido: (1) abrir `/health` da API; (2) entrar na web como médico; (3) abrir o Wokwi, iniciar a simulação e levar FC acima de 110 e SpO2 abaixo de 92; (4) conferir o risco ALTO no painel e no app em até 3 segundos; (5) na aba Triagem, escolher “Dor no peito”.

10. Evidências visuais

Capturas do sistema em funcionamento. Cada quadro cinza será substituído pelo print correspondente, salvo em docs/prints/ do repositório.

[inserir print]

Painel do médico (web)

[inserir print]

Detalhe em risco ALTO (web)

[inserir print]

Triagem com alerta de emergência

[inserir print]

Sensor no Wokwi com OLED em ALTO

[inserir print]

Aplicativo Android

[inserir print]

Aplicativo: triagem